

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

November 2024

Database development with PL/SQL

Reference reading



[11 Tuning PL/SQL Applications for Performance](#)

[PL SQL Performance Tuning](#)

[12 PL/SQL Optimization and Tuning](#)

[difference of explain plan, autotrace and tkprof](#)

[Youtube Tutorial: tkprof for trace analysis](#)

Lecture 11 – PL/SQL Performance Tuning

Objectives of PL/SQL Performance Tuning

Write Efficient PL/SQL Code	Learn techniques to improve execution speed and optimize existing PL/SQL programs.
Understand PL/SQL Optimization	Gain insights into how the PL/SQL compiler optimizes programs, including the role of optimization levels and when to adjust them.
Avoid Common Performance Issues	Identify poor practices in SQL and PL/SQL code and learn strategies to mitigate them, such as using indexes, statistics, and efficient SQL statements.
Optimize Loop Performance	Enhance the execution of loops with constructs like FORALL and BULK COLLECT, and reduce overhead in DML operations.
Improve Function Calls	Make subprograms and functions more efficient, especially those invoked within SQL queries, by techniques like caching, function-based indexes, and using NOCOPY.
Leverage Built-In Tools and Features	Use diagnostic tools like EXPLAIN PLAN and TKPROF for query analysis and tuning.
Minimize Overheads	Reduce unnecessary operations, such as implicit datatype conversions, redundant string functions, and unoptimized conditional tests.
Benchmark and Trace Performance	Profile programs to identify bottlenecks and use tracing tools to analyze runtime behavior for performance improvements.
Compile for Native Execution	Explore native compilation options to increase runtime efficiency for computationally intensive programs.
Utilize Advanced Features	Incorporate advanced PL/SQL features like dynamic SQL tuning, computation-intensive programming strategies, and optimized table functions to enhance application performance.

By focusing on these objectives, developers can systematically address and resolve performance challenges in PL/SQL applications.



PL/SQL Performance Tuning: Key Concepts and Strategies



Context Switching:

Minimize switching between PL/SQL and SQL execution to reduce overhead.

Cursor Management:

Use implicit cursors in loops to reduce explicit cursor handling.

Key Concepts

Bind Variables: Use bind variables to reuse execution plans and avoid excessive parsing.

Bulk Operations: Use BULK COLLECT and FORALL for processing multiple rows efficiently.

Minimize Context Switching:

Group SQL statements together and avoid unnecessary SELECT INTO operations.

Bulk Operations:

Use BULK COLLECT INTO and FORALL to process multiple rows in one operation.

Minimize DML in Loops:

Use FORALL to batch INSERT, UPDATE, or DELETE operations.

Optimize Loops:

Avoid row-by-row operations; use bulk processing to reduce overhead.

Tuning Strategies

Optimize SQL Queries:

Focus on efficient SQL queries, avoid SELECT *, and ensure proper indexing.

Indexing:

Create appropriate indexes on frequently queried columns to speed up lookups.

Examples illustrating the PL/SQL performance tuning concepts

Tuning Concept	Description	Example
Optimize SQL Queries	Focus on efficient SQL queries, avoid SELECT * , and specify only the needed columns.	Bad Practice: SELECT * FROM employees WHERE department_id = 10; Good Practice: SELECT employee_id, first_name, last_name FROM employees WHERE department_id = 10;
Indexing	Create appropriate indexes on frequently queried columns to speed up lookups.	CREATE INDEX idx_department_id ON employees(department_id); Usage: SELECT employee_id, first_name FROM employees WHERE department_id = 10;
NOCOPY Hint	Use NOCOPY for large parameters to avoid unnecessary memory copying.	PROCEDURE process_large_data(p_data IN OUT NOCOPY my_table_type) IS BEGIN -- Process data here END;
Transaction Management	Commit or roll back transactions promptly to avoid long-held locks and to maintain database integrity.	BEGIN -- Perform DML operations UPDATE employees SET salary = salary * 1.10 WHERE department_id = 10; -- Commit the transaction COMMIT; EXCEPTION WHEN OTHERS THEN -- In case of error, roll back the transaction ROLLBACK; END;

Examples illustrating the PL/SQL performance tuning concepts

Tuning Concept	Description	Example
Optimize Function Calls	Badly written subprograms (e.g., slow sort or search functions) harm performance. Avoid unnecessary calls to subprograms, and optimize their code.	Bad Practice: <i>FOR item IN (SELECT DISTINCT(SQRT(department_id)) FROM employees)</i> Calls the function for each row. Good Practice: <i>FOR item IN (SELECT SQRT(department_id) FROM (SELECT DISTINCT department_id FROM employees))</i> Calls the function once per distinct value, reducing overhead.
Function-Based Index	If a function is called within a SQL query, cache the function value for each row using a function-based index on the table.	<i>CREATE INDEX idx_sqrt_dept_id ON employees(SQRT(department_id));</i> This speeds up queries using the function.
Use NOCOPY for Parameters	Use NOCOPY for OUT or IN OUT parameters to avoid performance overhead when dealing with exceptions or large data.	<i>PROCEDURE process_data(p_data IN OUT NOCOPY my_table_type)</i> This avoids unnecessary copying when large collections or LOBs are passed back as parameters.
Self NOCOPY for Object Types	When modifying attributes of object types, declare the first parameter as SELF IN OUT NOCOPY to avoid copying all attributes at the end of a subprogram.	<i>TYPE employee_obj_type IS OBJECT (id NUMBER, name VARCHAR2(100));</i> <i>MEMBER PROCEDURE update_name (SELF IN OUT NOCOPY employee_obj_type, new_name VARCHAR2);</i> This avoids unnecessary copies of the object's attributes when the subprogram ends.

PL/SQL code for optimizing function calls

```
DECLARE
  item employees.department_id%TYPE;
BEGIN
  -- Inefficient: Calls SQRT for every row (slow for large tables)
  FOR item IN (SELECT DISTINCT(SQRT(department_id)) AS col_alias FROM
employees)
  LOOP
    dbms_output.put_line(item.col_alias);
  END LOOP;

  -- Efficient: Filters distinct department_id first, then calls SQRT only once per value
  FOR item IN
    (SELECT SQRT(department_id) AS col_alias
    FROM (SELECT DISTINCT department_id FROM employees))
  LOOP
    dbms_output.put_line(item.col_alias);
  END LOOP;
END;
/
```

Explanation

Inefficient:

- The SQRT(department_id) function is called for every row, which can be slow.

Efficient:

- The inner query selects distinct **department_id**, reducing rows.
- The outer query applies the **SQRT** function only once per distinct **department_id**. This speeds up performance by reducing function calls.

Further Optimization:

- Create a **function-based index** for **SQRT(department_id)** to speed up repeated queries:

```
CREATE INDEX idx_sqrt_dept_id ON employees (SQRT(department_id));
```



PL/SQL code for Optimizing



Note: In **Lecture 8**, we covered optimization techniques such as loops and bulk processing, focusing on how BULK COLLECT and FORALL can handle large datasets efficiently. In **Lecture 9**, we discussed pipelined functions for processing rows iteratively and object types for structured and reusable data representation. These approaches are essential for writing high-performance and maintainable PL/SQL code.



TKPROF and EXPLAIN PLAN



TKPROF Tool



TKPROF (Trace Kernel Profiler) is a utility that processes trace files generated by the SQL Trace facility. It formats the trace file data into a human-readable output and provides insights into SQL performance.

Features



- ☐ **Execution Plan:** Generates execution plans for traced SQL statements.
- ☐ **Statistics Storage:** Creates a SQL script for storing statistics in the database.
- ☐ **Analysis:** Identifies resource-intensive SQL statements and their efficiency.

Benefits



- ☐ Detect inefficient queries using **resource consumption statistics**.
- ☐ Combine trace files for a holistic view of database performance.
- ☐ Quickly locate issues impacting performance.

SQL Performance Analysis: Tracing and TKPROF Explained

The **SQL Trace facility** and **EXPLAIN PLAN** both aim to improve SQL performance, but they measure different things and serve different purposes in performance tuning:



SQL Trace (with TKPROF)

SQL Trace provides runtime performance data for individual SQL statements. It generates detailed statistics for each SQL statement that is executed, such as:

- **Parse, execute, and fetch counts:** How many times the SQL statement was parsed, executed, and fetched.
- **CPU and elapsed times:** The amount of CPU and time taken for the SQL execution.
- **Physical and logical reads:** The number of reads from disk (physical) and from the cache (logical).
- **Rows processed:** How many rows the SQL statement affected.
- **Library cache misses:** The number of misses when accessing the shared SQL area.
- **Commit and rollback counts:** The frequency of commit or rollback operations.

TKPROF is a tool used to analyze the trace file generated by SQL Trace, converting it into a human-readable format with execution plans, resource usage statistics, and other performance metrics.

Key Features:

- It measures **actual execution statistics**.
- Provides details about **I/O**, CPU usage, and other runtime aspects.
- Helps identify which SQL statements are using excessive resources.
- Can be used to see how frequently certain SQL statements are executed, and their resource utilization.

TKPROF Explained



EXPLAIN PLAN

EXPLAIN PLAN, on the other hand, is a static tool that shows the **execution plan** Oracle will use for a SQL statement before it is actually run. It explains how Oracle intends to execute the SQL statement, which operations will be used, and the order of execution (e.g., full table scan, index lookup, etc.).

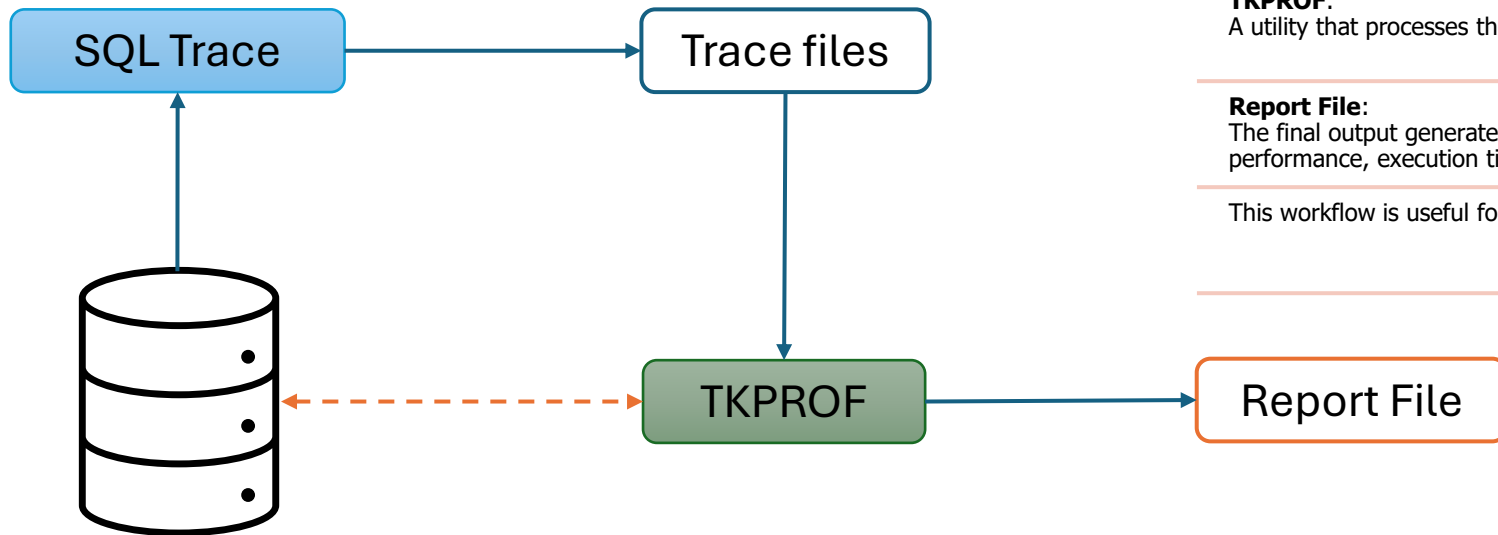
- **Execution Plan:** Shows the steps that Oracle plans to take to execute a query.
- **Access Methods:** Indicates how data will be retrieved (e.g., index scan, full table scan).
- **Join Methods:** Shows the type of joins used (e.g., nested loops, hash joins).
- You can run the EXPLAIN PLAN command before executing the query to see how the optimizer plans to execute the query, which helps identify inefficient access paths or operations (like full table scans when indexes are available).

Key Features:

- It provides **predictions of execution**, not actual runtime data.
- **Static**—it doesn't measure how long the query took, how much CPU or I/O was used, or how many rows were returned (except for estimated rows).
- **Helps in optimizing SQL statements** by analyzing the execution strategy Oracle uses

SQL Performance Analysis: Tracing Facility

The diagram illustrates the process of using SQL Trace and TKPROF for database performance analysis:



Database:

The source where SQL operations and queries are executed.



SQL Trace:

A tool that collects detailed statistics about SQL execution in the database. It monitors the performance of queries and writes this information to a **Trace File**.

Trace File:

A raw file containing detailed performance data and execution statistics generated by the SQL Trace facility. This file is typically difficult to interpret directly.

TKPROF:

A utility that processes the raw trace file and formats it into a human-readable **Report File**.

Report File:

The final output generated by TKPROF, containing clear and concise insights into SQL execution, such as query performance, execution time, and resource usage.

This workflow is useful for identifying and optimizing poorly performing SQL statements.

Using SQL Trace and TKPROF

Feature	SQL Trace (with TKPROF)	EXPLAIN PLAN
Purpose	Measures actual performance	Shows predicted execution plan
Data Collected	Actual execution statistics (e.g., CPU, I/O, rows processed)	Predicted execution plan (access methods, joins, etc.)
Usage	Used for performance tuning during runtime	Used for understanding the query execution strategy before execution
Granularity	Measures real resource usage per execution	Shows an estimated execution plan
Output Format	Trace file analyzed by TKPROF (including execution plan and stats)	Direct result showing execution plan steps

Enable SQL Trace

```
Command Prompt - sqlplus s X + v

C:\Users\Eric> sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA

SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 24 06:11:03 2024
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL>
SQL> ALTER SESSION SET sql_trace = TRUE;

Session altered.

SQL>
SQL> EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);

PL/SQL procedure successfully completed.

SQL>
```

- To connect to an Oracle database using SQL*Plus as the SYS user with SYSDBA privileges

C:\Users\Eric>sqlplus sys as SYSDBA

or

C:\Users\Eric> sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA

--Setting SQL_TRACE = TRUE for your session:
ALTER SESSION SET sql_trace = TRUE;

--This step is already covered by the command you used
EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE,
binds => TRUE);

SQL Trace and TKPROF (1/4)

```
SQL Plus
Windows PowerShell

SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 24 05:03:56 2024
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Enter user-name: system
Enter password:
Last Successful login time: Sun Nov 24 2024 04:57:14 +02:00

Connected to:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);

PL/SQL procedure successfully completed.

SQL> SELECT tracefile
2 FROM v$process p
3 JOIN v$session s ON p.addr = s.paddr
4 WHERE s.audsid = SYS_CONTEXT('USERENV', 'SESSIONID');

TRACEFILE
-----
C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_836
4.trc

SQL> |
```

Step 1: Enable SQL Trace for Your Session

--First, connect to your Oracle Database using SQL*Plus or any other SQL interface.

--To enable SQL trace for your session, execute the following command:
EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);
--This will start tracing for your session, capturing wait events and bind variables.

Step 2: Find Your Trace File

--To find the location of your trace file, run the following query:

```
SELECT tracefile
FROM v$process p
JOIN v$session s ON p.addr = s.paddr
WHERE s.audsid = SYS_CONTEXT('USERENV', 'SESSIONID');
```

--This query will return the path to your trace file.

SQL Trace and TKPROF (2/4)

```
Windows PowerShell
PS C:\Users\Eric> tkprof C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc output.txt explain=system/admin

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

PS C:\Users\Eric> cat .\output.txt

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Trace file: C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc
Sort options: default

*****
count      = number of times OCI procedure was executed
cpu        = cpu time in seconds executing
elapsed    = elapsed time in seconds executing
disk       = number of physical reads of buffers from disk
query      = number of buffers gotten for consistent read
current    = number of buffers gotten in current mode (usually for update)
rows       = number of rows processed by the fetch or execute call
*****

SQL ID: akjkx37y0xpm Plan Hash: 0

BEGIN DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE); END;

call      count          cpu    elapsed       disk    query    current    rows
-----
Parse         2         0.00         0.00         0         0         0         0
Execute       3         0.00         0.00         0         0         0         3
```

Step 3: Locate the Trace File in the File System

Navigate to the directory where the trace file is located (based on the output from the previous step). For example:

- C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace

Step 4: Use TKPROF to Format the Trace File

1. Open a command prompt or PowerShell window on your machine.
2. Navigate to the folder where the trace file is located, or directly provide the full path.
3. Run the following TKPROF command:

```
tkprof
C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc output.txt
explain=system/<password>
```

Step 5: View the Output File

After running the TKPROF command, you can view the output.txt file by using the following

```
cat .\output.txt
or
notepad .\output.txt
```

SQL Trace and TKPROF (3/4)

```
Windows PowerShell
PS C:\Users\Eric> tkprof C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc output.txt explain=system/admin

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

PS C:\Users\Eric> cat .\output.txt

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Trace file: C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc
Sort options: default

*****
count      = number of times OCI procedure was executed
cpu         = cpu time in seconds executing
elapsed     = elapsed time in seconds executing
disk        = number of physical reads of buffers from disk
query       = number of buffers gotten for consistent read
current     = number of buffers gotten in current mode (usually for update)
rows        = number of rows processed by the fetch or execute call
*****

SQL ID: akjkx37y0xpm Plan Hash: 0

BEGIN DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE); END;

call      count          cpu    elapsed       disk    query    current    rows
-----
Parse         2         0.00         0.00         0         0         0         0
Execute       3         0.00         0.00         0         0         0         3
```

Step 3: Locate the Trace File in the File System

Navigate to the directory where the trace file is located (based on the output from the previous step). For example:

- C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace

Step 4: Use TKPROF to Format the Trace File

1. Open a command prompt or PowerShell window on your machine.
2. Navigate to the folder where the trace file is located, or directly provide the full path.
3. Run the following TKPROF command:

```
tkprof
C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc output.txt
explain=system/<password>
```

Step 5: View the Output File

After running the TKPROF command, you can view the output.txt file by using the following

```
cat .\output.txt
or
notepad .\output.txt
```

SQL Trace and TKPROF (4/4)

```
output.txt
File Edit View

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Trace file: C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc
Sort options: default

*****
count      = number of times OCI procedure was executed
cpu         = cpu time in seconds executing
elapsed     = elapsed time in seconds executing
disk        = number of physical reads of buffers from disk
query       = number of buffers gotten for consistent read
current     = number of buffers gotten in current mode (usually for update)
rows        = number of rows processed by the fetch or execute call
*****

SQL ID: akjkyx37y0xpm Plan Hash: 0

BEGIN DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE); END;

call      count          cpu    elapsed       disk      query    current    rows
-----
Parse         2         0.00         0.00         0         0         0         0
Execute       3         0.00         0.00         0         0         0         3
Fetch         0         0.00         0.00         0         0         0         0
-----
total         5         0.00         0.00         0         0         0         3

Misses in library cache during parse: 0
Misses in library cache during execute: 1
Optimizer mode: ALL ROWS

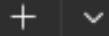
Ln 1, Col 1      57,369 characters
```

Step 6: Interpret the Results

- The output.txt file will contain performance details of the SQL statements executed during your session, including execution time, CPU usage, IO statistics, and any wait events.
- The EXPLAIN section will show you how the database is executing the SQL queries, which can be helpful for performance tuning.

EXPLAIN PLAN

Command Prompt - sqlplus s



```
C:\Users\Eric>sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA
```

```
SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 24 06:37:50 2024  
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Connected to:  
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production  
Version 21.3.0.0.0
```

```
SQL> DROP TABLE PLAN_TABLE;
```

```
Table dropped.
```

```
SQL> @?/rdbms/admin/utlxplan.sql
```

```
Table created.
```

- To connect to an Oracle database using SQL*Plus as the SYS user with SYSDBA privileges

```
C:\Users\Eric>sqlplus sys as SYSDBA
```

```
or
```

```
C:\Users\Eric> sqlplus  
sys/admin@localhost:1521/ORACLE21CDB AS  
SYSDBA
```

- Drop existing Plan_Table
SQL> DROP TABLE PLAN_TABLE;

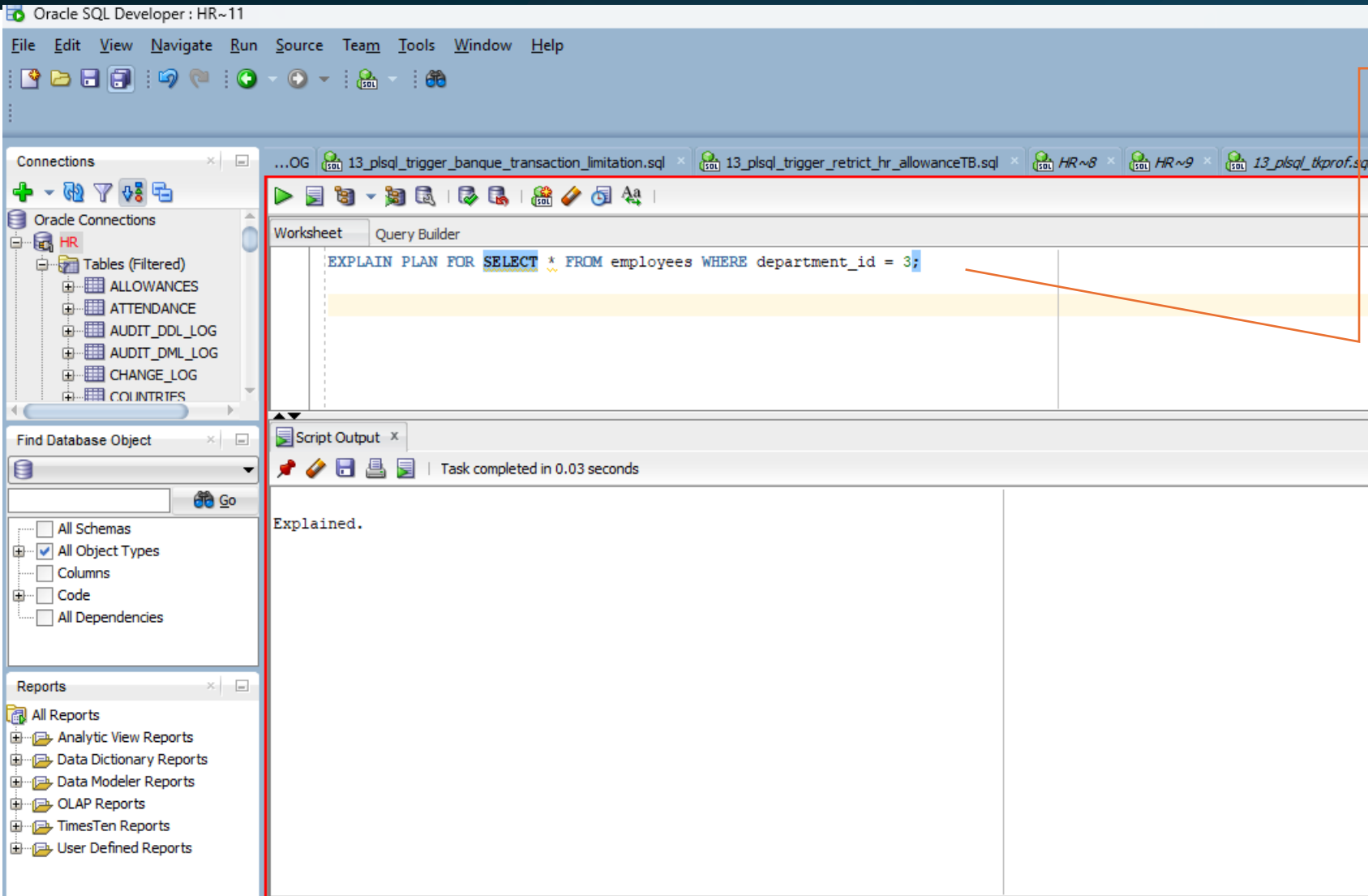
1.Prepare the Plan Table:

Use the default PLAN_TABLE provided by Oracle, or create one using the following:

```
*/
```

```
@?/rdbms/admin/utlxplan.sql
```

Generate Execution PLAN



2. Generate the Execution Plan: Use the EXPLAIN PLAN command:

*/

EXPLAIN PLAN FOR SELECT * FROM employees
WHERE department_id = 3;

View the Execution Plan

The screenshot shows the SQL*Plus interface with the following components:

- Menu Bar:** File, Edit, View, Navigate, Run, Source, Team, Tools, Window, Help.
- Connections Panel:** Shows Oracle Connections with a tree view including Tables (Filtered), ALLOWANCES, ATTENDANCE, AUDIT_DDL_LOG, AUDIT_DML_LOG, CHANGE_LOG, and COUNTRIES.
- Find Database Object Panel:** Includes a search bar and checkboxes for All Schemas, All Object Types, Columns, Code, and All Dependencies.
- Reports Panel:** Includes a search bar and checkboxes for All Reports, Analytic View Reports, Data Dictionary Reports, Data Modeler Reports, OLAP Reports, TimesTen Reports, and User Defined Reports.
- Worksheet:** Contains the SQL query: `EXPLAIN PLAN FOR SELECT * FROM employees WHERE department_id = 3;` and `SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());`.
- Script Output Panel:** Shows the execution plan results for the query.

Execution Plan Output:

```
PLAN_TABLE_OUTPUT
1 Plan hash value: 1445457117
2
3 -----
4 | Id | Operation          | Name          | Rows  | Bytes | Cost  | (%CPU) | Time  |
5 -----
6 | 0  | SELECT STATEMENT   |               |      1 |    28 |      3 |    (0) | 00:00:01 |
7 | *  | TABLE ACCESS FULL| EMPLOYEES     |      1 |    28 |      3 |    (0) | 00:00:01 |
8 -----
9
10 Predicate Information (identified by operation id):
11 -----
12
13      1 - filter("DEPARTMENT_ID">=3)
```



View the Execution Plan:: Query the PLAN_TABLE to see the plan details:

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());
```

Key Takeaways

1. The query scans all rows in the EMPLOYEES table, filtering for DEPARTMENT_ID = 3.
2. This is a relatively low-cost operation (cost = 3) since the table is small (28 rows).
3. For larger datasets, an index on DEPARTMENT_ID could improve performance.

All additional information or Partition details

Oracle SQL Developer : HR~11

File Edit View Navigate Run Source Team Tools Window Help

Connections

Oracle Connections

HR

Tables (Filtered)

- ALLOWANCES
- ATTENDANCE
- AUDIT_DDL_LOG
- AUDIT_DML_LOG
- CHANGE_LOG
- COUNTRIES

Find Database Object

Go

Reports

All Reports

- Analytic View Reports
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- TimesTen Reports
- User Defined Reports

Worksheet

Query Builder

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(NULL, NULL, 'ALLSTATS LAST'));
```

Script Output

Query Result

SQL | All Rows Fetched: 19 in 0.059 seconds

PLAN_TABLE_OUTPUT

```
1 Plan hash value: 1445457117
2
3 -----
4 | Id | Operation          | Name          | E-Rows |
5 |----|-----|
6 | 0 | SELECT STATEMENT    |               |        |
7 |*  1 | TABLE ACCESS FULL | EMPLOYEES     |      28 |
8 |----|-----|
9
10 Predicate Information (identified by operation id):
11 -----
12
13    1 - filter("DEPARTMENT_ID"=3)
14
15 Note
16 -----
17 - Warning: basic plan statistics not available. These are only collected when:
18   * hint 'gather_plan_statistics' is used for the statement or
19   * parameter 'statistics_level' is set to 'ALL', at session or system level
```



```
/*
To display additional information like predicates
or partition details:
*/
SELECT * FROM
TABLE(DBMS_XPLAN.DISPLAY(NULL, NULL,
'ALLSTATS LAST'));
```

Key Takeaways

- The query execution plan reveals a full table scan, which is fine for small tables but should be optimized for larger datasets.
- Enabling runtime statistics allows deeper analysis and tuning opportunities.
- Consider indexing frequently filtered columns like DEPARTMENT_ID to improve performance.

Summary: PL/SQL Performance Tuning

PL/SQL performance tuning optimizes code and database interactions to improve efficiency and reduce execution time. Key strategies include:

Efficient SQL: Use indexes, bind variables, and optimized queries (EXISTS over IN).



Bulk Processing: Replace row-by-row operations with BULK COLLECT and FORALL.

Minimize Context Switching: Reduce PL/SQL-to-SQL engine calls by consolidating operations.

Native Compilation: Compile PL/SQL into machine code for faster execution.

Profiling Tools: Use DBMS_PROFILER, DBMS_TRACE, and SQL Trace to identify bottlenecks.

Result Caching: Cache frequently accessed data or query results for reuse.

Loop Optimization: Avoid unnecessary iterations; use SQL within loops efficiently.

Built-in Functions: Leverage Oracle's built-ins for computations and transformations.

Testing & Monitoring: Continuously monitor performance using AWR, ADDM, and v\$ views.

Apply these techniques systematically for measurable improvements in performance.

Keep Learning!